

Persistent profiles and real browsers

Week 7, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we'll cover

- Headless vs visible browser: what changes, when each mode belongs
- What a browser profile directory contains and why it persists across runs
- `user_data_dir`: the one-line change that carries login state forward
- `storage_state` and `keep_alive`: lighter-weight alternatives
- Credentials and cookies: two separate secrets to protect

By the end of this lecture you can configure a persistent browser session, run a login agent once, and reuse the result in every subsequent agent run.

Headless vs visible

Two modes, one question

- **Headless** (default): no visible window; faster; needs no display server
- **Visible** (`headless=False`): a real window opens; slower; shows the agent working
- Anti-bot heuristics (Cloudflare, Akamai) treat the two modes differently
- For login flows that fail headless, switching to visible often unblocks them

```
from browser_use.browser import BrowserSession

session = BrowserSession(
    headless=False,
    window_size={"width": 1280, "height": 900},
)
```

Use headless in production. Use visible while building and debugging.

When visible mode is necessary

- The site uses a CAPTCHA that requires human intervention on first login
- Anti-bot detection flags the headless fingerprint
- You need to watch the agent work to understand why it is failing
- You are demonstrating the agent to a colleague

Visible mode requires a display. In a cloud VM, use Xvfb or a hosted browser service.

Profile directories

What a profile directory is

Every Chrome session writes state to a directory. With no profile path set, the directory is temporary. Set `user_data_dir` and it persists.

```
~/.bu_profiles/my-site/Default/  
├── Cookies           # SQLite: all cookies for all origins  
├── Local Storage/    # Web localStorage per origin  
├── IndexedDB/        # Per-origin databases  
└── Login Data        # Saved form credentials (encrypted)
```

The `Cookies` file is the key item. An HTTP session cookie written there by a successful login persists until the cookie expires.

Source: browser-use team, Browser Settings docs (2026)

Using user_data_dir

```
import pathlib
from browser_use.browser import BrowserSession

PROFILE = pathlib.Path.home() / ".bu_profiles" / "my-site"
PROFILE.mkdir(parents=True, exist_ok=True)

session = BrowserSession(
    user_data_dir=str(PROFILE),
    headless=False,
    keep_alive=False,
)
```

- Point two sessions at the same directory and they share cookies
- No login step required in the second session
- Works across separate Python processes and machine restarts

The two-agent pattern

```
# Agent 1: login once (run manually, or in a setup script)
login_agent = Agent(
    task="Go to example.com/login and sign in with x-user and x-pass.",
    browser_session=BrowserSession(user_data_dir=str(PROFILE)),
    sensitive_data={"x-user": "myname", "x-pass": "s3cret"},
    ...
)
await login_agent.run()

# Agent 2: task work, no credentials needed
task_agent = Agent(
    task="Go to example.com/dashboard and return the account balance.",
    browser_session=BrowserSession(user_data_dir=str(PROFILE)),
    ...
)
result = await task_agent.run()
```

storage_state and keep_alive

Two lighter-weight alternatives

storage_state: saves cookies and localStorage to a JSON file

```
# Save after a manual login
await context.storage_state(path="state.json")

# Restore in the next session
session = BrowserSession(storage_state="state.json")
```

keep_alive: leaves the browser running between agent calls (one process)

```
session = BrowserSession(keep_alive=True)
await agent1.run()    # logs in
await agent2.run()    # same browser context, same cookies
await session.close()
```

`user_data_dir` is the most durable (survives restarts). Use `keep_alive` for a short pipeline in one Python process.

Credentials and cookies: two secrets to
protect

The threat model has two layers

What	Risk	Mitigation
Username + password	Leaks into LLM context	<code>sensitive_data</code> placeholders
Session cookies	Leaks to filesystem or VCS	<code>chmod 700, .gitignore</code>

Both protections are always required for authenticated agents.

- `sensitive_data` ensures the LLM sees only placeholder names, not the real credentials
- Profile directory must be outside version control and readable only by your user

Scope restriction with allowed_domains

```
session = BrowserSession(  
    user_data_dir=str(PROFILE),  
    allowed_domains=["app.example.com"],  
)
```

- The agent cannot navigate to any domain outside this list
- A prompt injection attack that redirects to a different domain is blocked
- Narrow the list to the minimum domains the task requires

Source: browser-use team, Browser Settings docs (2026)

Take-aways

1. `headless=False` opens a visible browser; use it for login flows that fail headless and for debugging.
2. `user_data_dir` points the browser at a persistent directory; the session cookie written by a login is reused in the next run.
3. The two-agent pattern separates login (once, manually) from task work (every run, no credentials).
4. `sensitive_data` keeps credentials out of the LLM context; `chmod 700` and `.gitignore` keep session cookies off the network and out of version control.
5. `allowed_domains` limits where the agent may act, the first line of defense against prompt injection.

"user_data_dir persists cookies and login state across runs"

What's next

- **Lecture 2 (this week):** Multi-tab work, downloads, and recording runs for debugging
- **Section (this week):** Persist a login once; run a second agent across two tabs using the same session
- **Reading:** Week 7 reading at `/c/browser-use-26su/readings/wk07`
- **Capstone out this week:** assigned now, due week 10